

Deep Neural Networks for Object Detection

Teams

Work individually or in teams of 2 students.

You should feel free to get general advice (e.g. getting the code to work, Ubuntu, compiling Darknet, C++, etc.) from other students or the Internet, but your answers to the problems should be your own.

Submit:

- a ZIP file containing your source code;
- a PDF report containing your answers to the questions below.

Images and tables should be included inline with the text in the PDF.

This assignment should take approximately 4 hours. The Darknet setup and supplied code should normally be working in less than one hour.

Goals

- Exposure to YOLO-based convolutional neural networks for object detection
- Experience using YOLOv3 and YOLOv7
- Understanding object-detection output
- Understanding Intersection over Union (IoU)
- Insight into comparing object-detection algorithms
- Insight into strengths and weaknesses of YOLOv3 and YOLOv7

Darknet: setup and simple usage

For this assignment, we will use the current Darknet V5 implementation:

<https://codeberg.org/CCodeRun/darknet>

The assignment has been tested using:

```
Darknet V5 "Moonlit"  
v5.1-104-gf684e1d7  
commit f684e1d75d4594298c8e73acc727ba6cf2e81c60  
Ubuntu 26.04  
CPU-only execution
```

A GPU is not required for this assignment.

You will compare two pretrained object-detection networks:

```
YOLOv3  
YOLOv7
```

Both networks use the 80 COCO object categories.

The following Darknet files will be used:

```
cfg/yolov3.cfg  
cfg/yolov7.cfg  
cfg/coco.names
```

The corresponding pretrained weights will also be provided or linked from the course page.

A typical object-detection result contains:

```
class  
confidence  
x  
y  
width  
height
```

For example:

```
class=dog confidence=0.99 x=123 y=223 w=197 h=321
```

This means the network predicts a dog with confidence 0.99 at the specified bounding box.

For this homework you will be given a C++ program that already performs:

- Darknet/model loading;
- image loading;
- object detection;
- inference-time measurement;
- ground-truth loading;
- matching predicted objects with ground-truth objects;
- calculation of True Positives, False Positives and False Negatives;
- calculation of Precision and Recall;
- processing of all supplied images.

You only need to complete the small IoU function described in Problem 1.

Installing Darknet on Your System

Install gcc, g++, CMake, OpenCV and the other dependencies:

```
sudo apt-get install build-essential git libopencv-dev cmake libprotobuf-dev  
protobuf-compiler
```

Install OpenBLAS for CPU execution:

```
sudo apt-get install libopenblas64-0 libopenblas64-0-openmp libopenblas64-  
openmp-dev
```

Download Darknet:

```
mkdir -p ~/src
cd ~/src

git clone https://codeberg.org/CCodeRun/darknet.git

cd darknet
git checkout f684e1d75d4594298c8e73acc727ba6cf2e81c60
```

Compile Darknet:

```
mkdir build
cd build

cmake -DCMAKE_BUILD_TYPE=Release ..
make -j4 package
```

Install the generated package:

```
sudo dpkg -i darknet-5.1.104-Linux.deb
```

Check that Darknet works:

```
darknet version
```

For CPU-only systems the output should indicate that GPU support is disabled.

You will also be given:

```
starter.cpp

dataset/
  images/
  ground_truth/
  selected_images.txt
```

The supplied dataset contains 15 selected images from the COCO validation dataset.

The images are divided into:

```
5 relatively easy images
5 medium images
5 difficult images
```

These groups are defined only for this homework and are based roughly on the number, size and complexity of the annotated objects.

Compile the supplied program with:

```
g++ -std=c++17 starter.cpp -o detector \
    -ldarknet $(pkg-config --cflags --libs opencv4)
```

Ground Truth and Evaluation Measures

Ground Truth:

The ground truth is the reference annotation describing which objects are actually present in an image and their correct classes and bounding boxes. Ground-truth annotations for all 15 supplied images are included in dataset/ground_truth/, alongside the supplied images in dataset/images/.

True Positive (TP):

A predicted object that is matched correctly to a ground-truth object of the same class and satisfies the required IoU threshold.

False Positive (FP):

A predicted object that is not successfully matched to a ground-truth object.

False Negative (FN):

A ground-truth object that the network failed to detect correctly.

Precision:

The fraction of predicted detections that are correct: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$.

Recall:

The fraction of ground-truth objects that are detected correctly: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$.

Mean IoU:

The average IoU of the correctly matched detections.

Average inference time:

The average time required by the network to process one image.

For this assignment, a prediction is matched only when the predicted class is the same as the ground-truth class and the IoU satisfies the threshold specified in Problem 1.

Problems

Please have a direct answer to each question - label the answers clearly.

(1a)

Complete the following function in the supplied C++ code:

```
double calculate_iou(  
    const cv::Rect2d &a,  
    const cv::Rect2d &b)  
{  
    // your code  
}
```

Intersection over Union (IoU) measures the overlap between two bounding boxes:

$$\text{IoU} = \text{area of intersection} / \text{area of union}$$

An IoU of 1.0 means that the two boxes overlap perfectly.

An IoU of 0.0 means that the two boxes do not overlap.

For this assignment, a prediction is considered a correct detection when:

$$\text{predicted class} = \text{ground-truth class}$$

and

$$\text{IoU} \geq 0.5$$

In your report, state the minimum IoU required for a correct detection and briefly explain what happens when the IoU is below this value.

(1b)

Run the supplied evaluator using YOLOv3 and YOLOv7 on the 15 supplied images.

For each model, report:

- average inference time;
- True Positives (TP);
- False Positives (FP);
- False Negatives (FN);
- Precision;
- Recall;
- Mean IoU of the correctly matched detections.

Show your results in a table.

Table 1. YOLOv3 and YOLOv7 Results

	Average Inference Time	TP	FP	FN	Precision	Recall	Mean IoU
YOLOv3							
YOLOv7							

(1c)

Which network is faster on your system?

Approximately how much faster is it?

(1d)

Which network obtains the higher Precision?

Which obtains the higher Recall?

Which obtains the higher Mean IoU?

(1e)

Show at least one example of an object detected correctly by both YOLOv3 and YOLOv7.

Show one example of an object detected correctly by YOLOv3 but not YOLOv7, if you find one.

Show one example of an object detected correctly by YOLOv7 but not YOLOv3, if you find one.

(1f)

Are there any discernible patterns in the differences between YOLOv3 and YOLOv7?

(2)

The supplied images are divided into three groups:

easy
medium
difficult

Run the supplied evaluator and report the Recall obtained by YOLOv3 and YOLOv7 for each group.

Show the results in a table.

Table 2. Recall Per Difficulty Group

	Easy	Medium	Difficult
YOLOv3			
YOLOv7			

(2a)

Does Recall change as the images become more difficult?

(2b)

Show two images where one or both networks perform poorly.

(2c)

Discuss possible reasons for the poor detections.

For example, consider:

- small objects;
 - occlusion;
 - clutter;
 - multiple nearby objects;
 - unusual viewpoints.
-

(3)

Discuss and comment on the strengths and weaknesses of YOLOv3 and YOLOv7 based on your results above.

Your discussion should consider:

- inference speed;
- Precision;
- Recall;
- Mean IoU;
- False Positives;
- False Negatives;
- performance on simple and difficult images.

Does the newer YOLOv7 model necessarily perform better than YOLOv3 in your experiments?

Explain your answer based on your experimental results.

Finally, explain why results from this small 15-image dataset should not be interpreted as proving that one model is always better than the other.

If you are on a LIACS PC room computer

The LIACS Linux Workstation (LLW) computers use Ubuntu 24.04 and provide the required compiler and libraries through environment modules. Do not use `apt-get` or `sudo dpkg -i` on a LIACS workstation. Use the following setup instead.

Load the required modules:

```
module purge
module load OpenCV/4.8.1-foss-2023a-CUDA-12.1.1-contrib
module load protobuf/24.0-GCCcore-12.3.0
module load CMake/3.26.3-GCCcore-12.3.0
module load Doxygen/1.9.7-GCCcore-12.3.0
```

Build the tested Darknet revision locally in your home directory:

```
mkdir -p ~/src
cd ~/src
git clone https://codeberg.org/CCodeRun/darknet.git darknet_liacs
cd darknet_liacs
git checkout f684e1d75d4594298c8e73acc727ba6cf2e81c60

mkdir -p build
cd build
cmake \
  -DCMAKE_BUILD_TYPE=Release \
  -DDARKNET_TRY_CUDA=OFF \
  -DDARKNET_TRY_ROCM=OFF \
  -DDARKNET_TRY_OPENBLAS=OFF \
  ..
cmake --build . -j4
```

OpenBLAS is deliberately disabled on the LIACS workstations because the available LIACS OpenBLAS installation uses a different 64-bit CBLAS interface from the one expected by this Darknet revision. This does not affect the correctness of the assignment; Darknet runs normally in CPU-only mode.

Verify the local build:

```
LD_LIBRARY_PATH="$HOME/src/darknet_liacs/build/src-lib:${LD_LIBRARY_PATH:-}" \
"$HOME/src/darknet_liacs/build/src-cli/darknet" version
```

The output should report Darknet V5 "Moonlit" v5.1-104-gf684e1d7 and CPU execution.

Compile the supplied program on LIACS

```
g++ -std=c++17 starter.cpp -o detector \
-I"$HOME/src/darknet_liacs/src-lib" \
-L"$HOME/src/darknet_liacs/build/src-lib" \
-Wl,-rpath,"$HOME/src/darknet_liacs/build/src-lib" \
-ldarknet \
$(pkg-config --cflags --libs opencv4)
```

For the YOLOv3 and YOLOv7 commands, use the configuration files in `~/src/darknet_liacs/cfg/` instead of `~/src/darknet/cfg/`. All other assignment steps are unchanged.

Remote access to a LIACS workstation

The same LIACS PC room computers can be used remotely through the LIACS SSH gateway. For example:

```
ssh YOUR_ULCN@0065066.student.liacs.nl \
-J YOUR_ULCN@ssh.liacs.nl
```

`0065066.student.liacs.nl` is only an example. Choose any powered-on LIACS Linux Workstation from the current computer list at <https://rel.liacs.nl/LLW>. If a machine is turned off, choose another hostname from the list.

Use `ssh.liacs.nl` as the jump host. The LIACS lab-room computers are not directly reachable through the ISSC gateway `sshgw.leidenuniv.nl`. After connecting remotely, follow the same LIACS setup commands above.

This LIACS setup was tested end-to-end on a Gorlaeus LIACS Linux Workstation running Ubuntu 24.04.5 in CPU-only mode.